since part of my CV works depends on CCTV camera connections and networks, I often face problem finding and connecting the camera conncted to my system. I normally get rtsp url from the camera then put them in my system. but for client, this isnt easy. client wants, I will put camera IP, user name and password adn then click connect. also, finding caemra IP is naothr problem. I wanto to scan. also, when it's not the same network, it can not find the camera and connect. I want to have a system for this part. No matter in which network cmaera lives, which gates, port or whatever, my system should scan for the cameras, get IP addresses. construct rtsp itself depeneding on the cameera vendor. Are these all possible?

Yes. **Most of what you described is possible.** However, there is one important networking limitation that no software can bypass.

The problem can be divided into **five independent problems**.

---

1. Camera Discovery (Same LAN)

100% possible.

This is exactly how software like

- Hikvision SADP
- Dahua ConfigTool
- ONVIF Device Manager
- Milestone
- Nx Witness
- Blue Iris

find cameras.

They use combinations of

- ONVIF Discovery (WS-Discovery)
- SSDP
- mDNS
- Vendor discovery protocols
- ARP scanning
- ICMP Ping
- TCP port scanning

Instead of asking the user

```
RTSP:
rtsp://192.168.1.64:554/Streaming/Channels/101
```

they simply show

```
✓ Hikvision Camera
192.168.1.64

Connect
```

---

2. Construct RTSP Automatically

Also possible.

Every vendor has several standard RTSP paths.

For example

Hikvision

```
rtsp://user:pass@ip:554/Streaming/Channels/101
```

or

```
.../102
```

Dahua

```
rtsp://user:pass@ip:554/cam/realmonitor?channel=1&subtype=0
```

Uniview

```
rtsp://user:pass@ip:554/media/video1
```

Axis

```
rtsp://user:pass@ip:554/axis-media/media.amp
```

So your software can

```
Find camera

↓

Identify vendor
```

```
↓

Generate correct RTSP

↓

Try connection

↓

Done
```

The client never sees RTSP.

---

3. Detect Camera Vendor Automatically

Possible.

Several methods.

ONVIF

Returns

```
Manufacturer

Model

Firmware

Serial
```

Example

```
Manufacturer:
HIKVISION

Model:
DS-2CD2043G0
```

Or HTTP headers

```
Server:

Hikvision-Webs
```

or

```
Dahua
```

Or MAC Address

The first bytes indicate vendor.

Example

```
00:23:63
```

```
↓
```

```
Hikvision
```

Very common.

---

4. Test Username & Password

Possible.

Workflow

```
Camera found
```

```
↓
```

```
Try ONVIF login
```

```
↓
```

```
Success?
```

```
↓
```

```
Generate RTSP
```

```
↓
```

```
Open stream
```

If login fails

```
Wrong password
```

instead of

```
Cannot connect
```

Much better UX.

---

5. Scan Cameras Across Different Networks

This is where physics and networking rules apply.

If the camera is

```
192.168.0.x
```

and your PC is

```
10.0.0.x
```

without any route between them...

It is impossible.

No software can magically discover devices on a network it cannot reach.

Example

```
Your laptop

↓

WiFi A

↓

Internet

------------------------

Factory

↓

Private LAN

↓

Camera
```

Your laptop cannot broadcast

```
Who is there?
```

into the factory LAN.

Broadcast packets never cross routers.

So discovery only works if

- same subnet
- VPN
- routed network
- discovery relay
- cloud gateway

How Professional VMS Systems Solve This

Enterprise VMS systems don't try to discover every camera from the client PC. Instead, they deploy a component inside each site.

```
          Internet
              │
     ┌────────┴────────┐
     │                 │
 Dashboard        Cloud Server
     │                 │
     └────────┬────────┘
              │
       Secure Connection
              │
     Site Gateway / Agent
              │
     ┌────────┴─────────┐
     │                  │
  Camera A          Camera B
```

The local gateway:

- scans the LAN,
- discovers cameras,
- identifies vendors,
- validates credentials,
- constructs RTSP URLs,

- reports everything back to your central system.

This architecture works across NAT because the gateway initiates an outbound connection to your server.

If There Is No ONVIF

Some cheap cameras lack ONVIF.

You can still try:

```
Ping

↓

Open TCP 554

↓

Open TCP 80

↓

Read HTTP page

↓

Guess vendor

↓

Try known RTSP templates

↓

Open stream
```

Many systems do exactly this.

I Would Build It Like This

Given your background with Jetson, DeepStream, and enterprise CV deployments, I would treat camera management as a dedicated subsystem rather than scattering connection logic throughout the application.

```
Camera Manager

├── Network Scanner
│      ├── ARP
│      ├── Ping
│      ├── ONVIF Discovery
│      ├── SSDP
│      └── mDNS
│
├── Vendor Detector
│      ├── ONVIF
│      ├── HTTP
│      ├── MAC OUI
│      └── Fingerprinting
│
├── Credential Tester
│
├── RTSP Generator
│
├── Stream Validator
│
├── Snapshot Generator
│
├── Health Monitor
│
└── Camera Registry
```

Then your user interface becomes as simple as:

```
Scan Network

↓

Found 12 Cameras

✓ Hikvision DS-2CD2043
✓ Dahua IPC-HDW2431
✓ Uniview IPC3614

↓

Select

↓

Username

Password

↓

Connect

↓

Done
```

No RTSP URLs.

No vendor-specific knowledge.

A further evolution: a "Universal Camera Connector"

This could become a standalone product that every one of your CV applications uses. Beyond discovery and connection, it could:

- Continuously monitor camera health and alert when a camera goes offline or credentials change.
- Benchmark stream latency, FPS, resolution, codec (H.264/H.265), and bitrate.
- Manage firmware and capability information (ONVIF profiles, PTZ support, audio, events).
- Expose a single API to your applications, so they request `"camera_id: warehouse_gate_3"` instead of dealing with IPs or RTSP strings.

For your long-term vision of deploying computer vision systems across many customer sites, this abstraction layer would eliminate much of the repetitive integration work and make your applications largely vendor-agnostic.